

个人背景、当前项目与后续方向整理

版本：v1.0 用途：个人技术档案 / 项目总结 / 方向规划

一、个人背景

1. 总体定位

你属于典型的**机器人系统工程型开发者**，关注重点不是单点算法，而是机器人系统从底层执行、通信、感知到上层决策的完整闭环。你的技术路线具有很强的工程导向，核心目标是把真实机器人系统稳定落地，而不是停留在概念验证层面。

你的长期关注方向包括：

- 分布式机器人控制系统
- STM32 / CAN / FreeRTOS 底层控制
- ROS2 机器人系统集成
- 多传感器融合与状态估计
- 边缘AI感知节点部署
- 强化学习与仿真到实物迁移
- 移动机器人与未来机械臂/四足方向

2. 技术风格

你的技术风格非常明确：

- 偏重系统工程，而不是纯理论推导
- 注重真实硬件验证和可运行性
- 喜欢将复杂系统拆分为可维护的模块
- 强调分层架构、通信解耦与职责清晰
- 更在意“系统能否稳定工作”，而不是“单个模块是否漂亮”

3. 已形成的能力结构

底层嵌入式与实时控制

你已经长期接触并实践：

- STM32 外设开发
- UART / CAN / I2C / SPI / TIM
- PID 闭环控制
- 编码器接入与调试
- 电机节点控制
- FreeRTOS 任务结构设计
- 分布式控制节点划分

这意味着你已经具备较强的底层控制器开发能力，而不是停留在简单外设使用层面。

机器人通信与中间件

你已经搭建并熟悉：

- CAN 总线通信
- 串口转 CAN 架构
- SocketCAN / slcand / SavvyCAN 调试体系
- ROS2 与底层节点的联动思路
- 未来工业以太网/QoS 数据骨干的规划

这说明你已经进入机器人系统中的“通信中枢层”建设阶段。

感知与定位

你已经接触并实践：

- IMU
- 磁力计
- 激光雷达
- 深度相机
- 点云可视化
- RTAB-Map
- 传感器融合与状态估计

这说明你已经从控制工程扩展到感知与定位系统，开始构建机器人对环境与自身状态的估计能力。

边缘AI与视觉部署

你已经具备 Jetson Nano 的工程实践经验，并开始建立：

- Docker 化部署流程
- TensorRT 加速推理管线
- ONNX 转换与模型部署
- 视觉推理节点化思路

这意味着你已经从“会跑模型”迈向“会把模型部署进机器人系统”。

强化学习与仿真

你已经开始接触并实践：

- MuJoCo
- UniLab
- SAC / APPO / FastSAC
- 四足机器人训练
- sim-to-real 迁移思路

这说明你正在从传统控制进一步进入现代机器人学习控制范式。

二、当前项目总结

1. 舵轮底盘项目

这是你当前非常核心的项目之一。

项目结构

你的舵轮底盘采用对角线布置，由两个轮组构成，每个轮组包含：

- 一个驱动电机
- 一个舵向电机

每个 STM32F103 节点控制一个轮模块，主控通过串口接收外部命令，再通过 UART 转 CAN 下发控制指令。

当前工程重点

你当前关注的核心问题包括：

- 舵向与驱动的协同控制
- 舵轮方向不一致问题
- 多电机控制与指令组织
- CAN 信息一次发送多条的实现方式
- 正运动学 / 逆运动学在底盘上的落地
- PID 闭环与状态反馈
- 底盘运动控制与模块状态解耦

当前阶段判断

这个项目已经不再是“能不能动”的问题，而是进入了：

- 运动学映射是否正确
- 模块方向是否统一
- 控制结构是否清晰
- 通信是否高效可靠
- 实机运行是否稳定

这类典型的系统工程阶段。

2. CAN 分布式控制项目

这是你底层架构最重要的一条主线。

核心设计思路

- 不让单一主控承担所有实时电机控制
- 通过多个 MCU 节点分担控制任务
- 主控负责高层指令与调度
- 各节点负责本地闭环控制

这套架构的优势

- 实时性更好
- 故障隔离更强
- 系统扩展性更高
- 适合多电机并行控制
- 符合机器人分层控制架构

当前关注的问题

- 如何组织多条 CAN 报文
- 如何设计统一的数据结构
- 如何用队列/结构体组织任务间通信
- 如何减少耦合，提高可维护性

你已经进入真正的系统设计阶段，而不是仅仅写单片机驱动代码。

3. ROS2 机器人系统集成项目

你已经在把底层控制、传感器和感知统一纳入 ROS2 系统。

已完成或正在进行的内容

- ROS2 Humble 部署
- Ubuntu 20.04 / 22.04 环境适配
- Raspberry Pi 5 / Jetson Nano / 虚拟机多平台验证
- Astra Pro 接入
- VLP-16 接入
- RealSense D435i 接入
- RViz 显示
- RTAB-Map 初步运行

当前目标

你现在关注的不是“ROS2 是什么”，而是：

- 如何把底层节点标准化
 - 如何把 CAN 节点抽象成 ROS2 接口
 - 如何形成 `cmd_vel` → `kinematics` → `CAN` → `motor feedback` 的闭环
 - 如何把系统做成长期可维护的工程结构
-

4. 边缘AI项目

你已经把 Jetson Nano 明确定位为一个边缘 AI 节点，而不是主控。

你已经在做的内容

- Docker 容器化部署
- TensorRT 推理加速
- ONNX 模型转换

- YOLO 推理流程验证
- SSD-MobileNet 基线测试

架构意义

这代表你已经开始建设一个较为清晰的机器人计算架构：

- Jetson 负责视觉推理
- ROS2 负责信息分发
- STM32 负责实时执行
- 主控负责系统协调

这是一种很合理的机器人边缘计算分层方式。

5. 多传感器融合与定位项目

你已经具备了形成统一状态估计链路的基础。

现有传感器基础

- IMU
- 磁力计
- 激光雷达
- RGBD 相机
- 编码器

后续自然方向

- 轮速里程计
- 姿态融合
- 轮速 + IMU 融合
- LiDAR-IMU 融合
- RGBD / 点云辅助定位
- EKF / UKF 状态估计

这一部分的目标不是堆叠传感器，而是建立稳定可靠的状态估计系统，为控制与导航提供可信输入。

6. 强化学习与四足机器人项目

你已经开始将 RL 与机器人实体控制结合。

当前实践方向

- MuJoCo 仿真
- UniLab 框架
- SAC / APPO / FastSAC
- 四足机器人 12-DOF 训练
- sim-to-real 迁移

这个方向的意义

它会把你的能力从传统控制推进到现代机器人控制方法，包括：

- 策略控制
- 奖励设计
- 接触动力学理解
- 仿真迁移
- 行为策略与约束处理

三、后续方向

1. 优先目标：把舵轮底盘做成完整闭环样板工程

这是最优先的方向。

目标链路

上位机命令 → 运动学计算 → CAN 下发 → 舵向/驱动闭环 → 反馈 → 里程计/状态更新

这条链路的重要性

它是你的机器人系统工程样板。只要这条链路稳定跑通，后面无论接：

- 导航
- 视觉
- SLAM
- RL
- 机械臂

都会有统一底层基础。

2. 标准化底层通信与任务架构

你后续需要更明确以下内容：

- 哪些数据走 CAN
- 哪些数据走 UART
- 哪些数据走 ROS2 topic
- 哪些数据进入队列
- 哪些任务是实时任务
- 哪些任务是非实时任务

这会直接决定整个系统的可维护性和扩展性。

3. 建立统一状态估计链

这是你从“会控制”走向“会导航”的关键步骤。

重点目标

- 轮速里程计
- IMU 姿态估计
- 磁力计航向参考
- 雷达/视觉辅助修正
- 更稳定的位姿估计

本质目标是：让机器人不仅知道“自己在动”，还尽量知道“自己在哪里”。

4. 固化 Jetson Nano 为边缘 AI 节点

你已经有很明确的方向，后续应进一步固定角色划分：

- Jetson：视觉推理/感知增强
- Raspberry Pi 5 或上位机：ROS2 中枢与任务协调
- STM32：实时控制与执行

这种分工会让系统结构更加清晰，也便于后续替换硬件。

5. 将 RL 从实验转为可接入的控制策略组件

你后续不应只做训练，而应进一步思考：

- 哪些任务适合 RL
- 哪些任务适合经典控制
- 哪些任务适合混合控制
- RL 策略如何安全落地
- 如何与底层控制器配合
- 如何做边界约束与保护

比较现实的路径是：

1. 仿真验证
 2. 局部策略接入
 3. 小范围 sim-to-real
 4. 再逐步提升复杂度
-

四、综合判断

你当前已经不属于单纯学习阶段，而是进入了**系统拼装与工程整合阶段**。

你的主要优势在于：

- 能把硬件、通信、控制、感知串联起来
- 有较强的系统拆解能力
- 重视真实运行与工程闭环
- 有明确的机器人长期方向

- 正在构建统一而连续的技术栈

你的未来竞争力，很大程度上会体现在：

■ 能否把一个复杂机器人系统真正稳定地做出来、跑起来、持续迭代下去。

五、压缩版结论

你是一名以机器人系统工程为核心方向的开发者，已经建立了从底层电机控制、CAN 分布式通信、ROS2 系统集成，到边缘 AI、传感器融合、强化学习控制的完整技术栈雏形。当前最重要的任务，是把舵轮底盘、通信架构、状态估计和边缘感知整合成一个稳定的样板系统；后续则可以逐步向导航、智能感知和 RL 控制融合方向扩展。